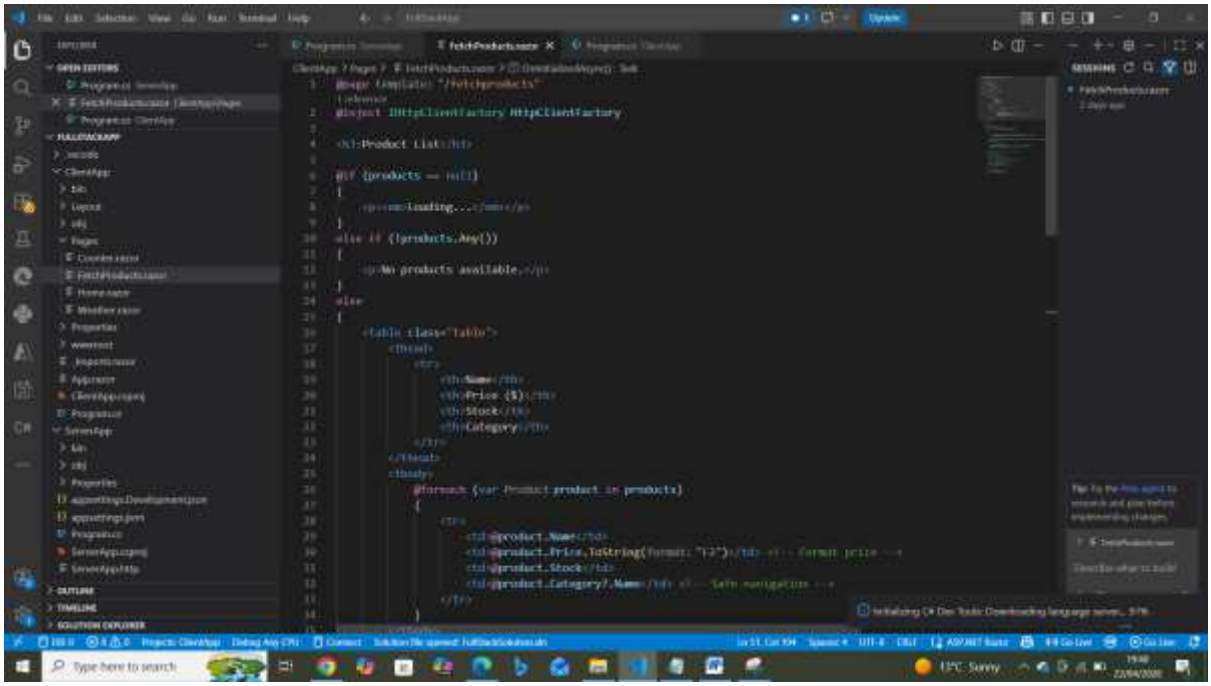
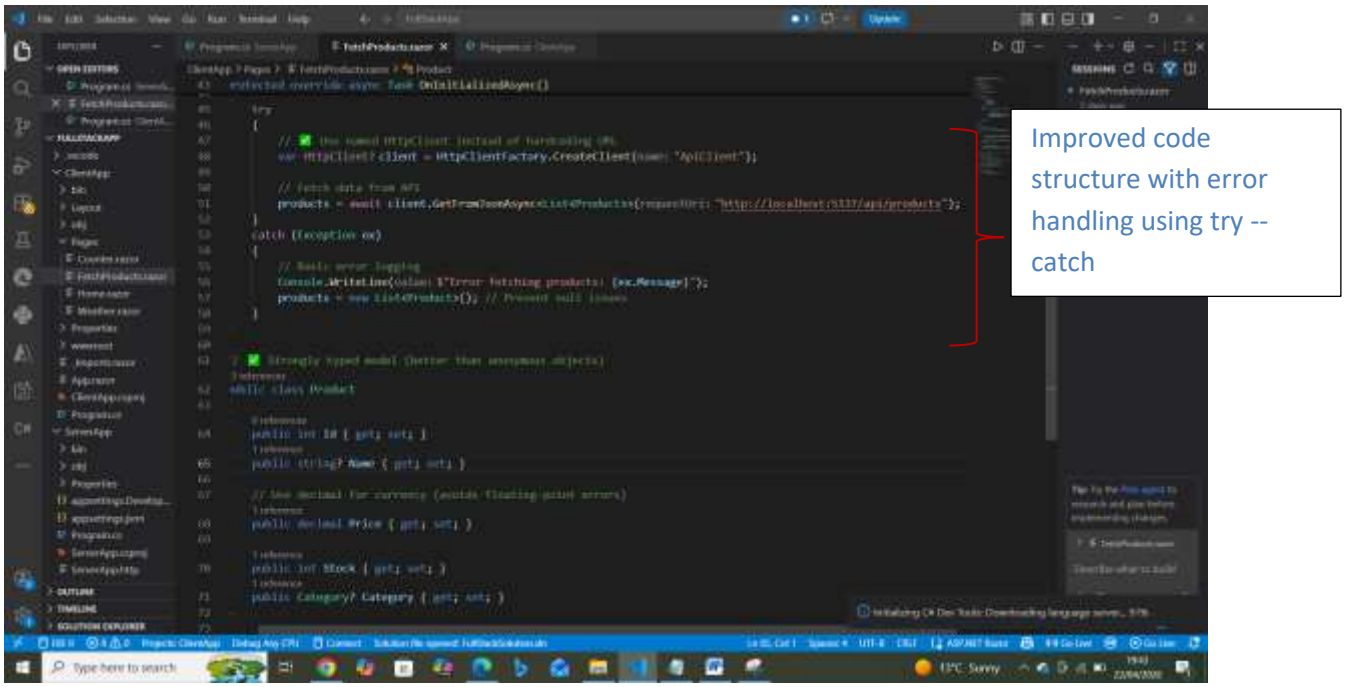
Candidate Name: Kayode Oladipo

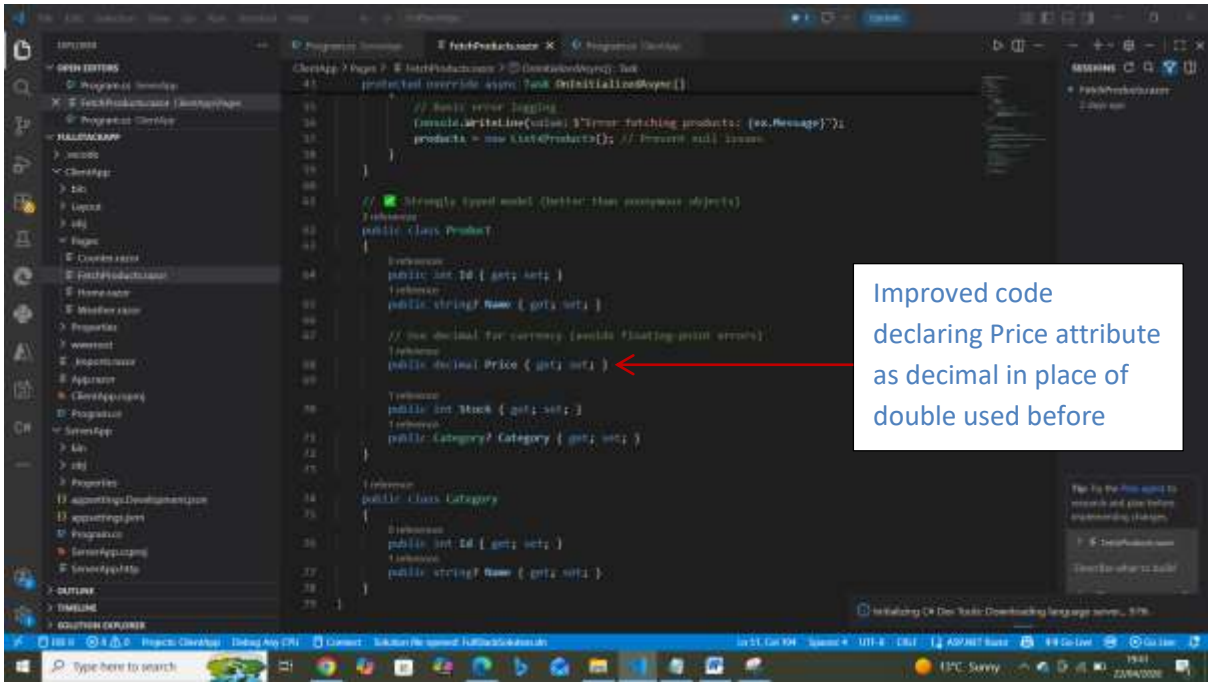
INVENTORY HUB APPLICATION

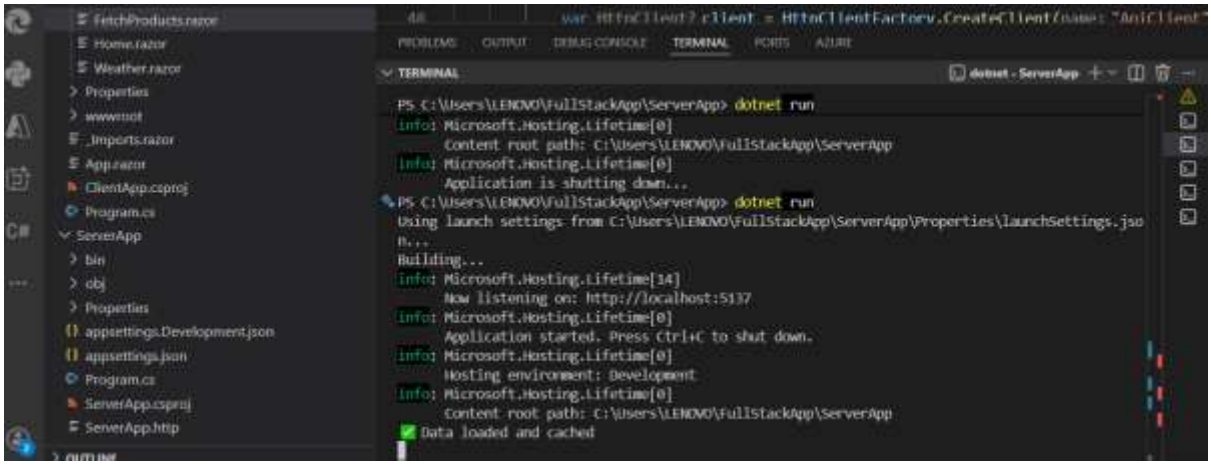
A table has been used in the Blazor frontend to display the data. A model for the data was also created here and asynchronous programming was used here to fetch the data.





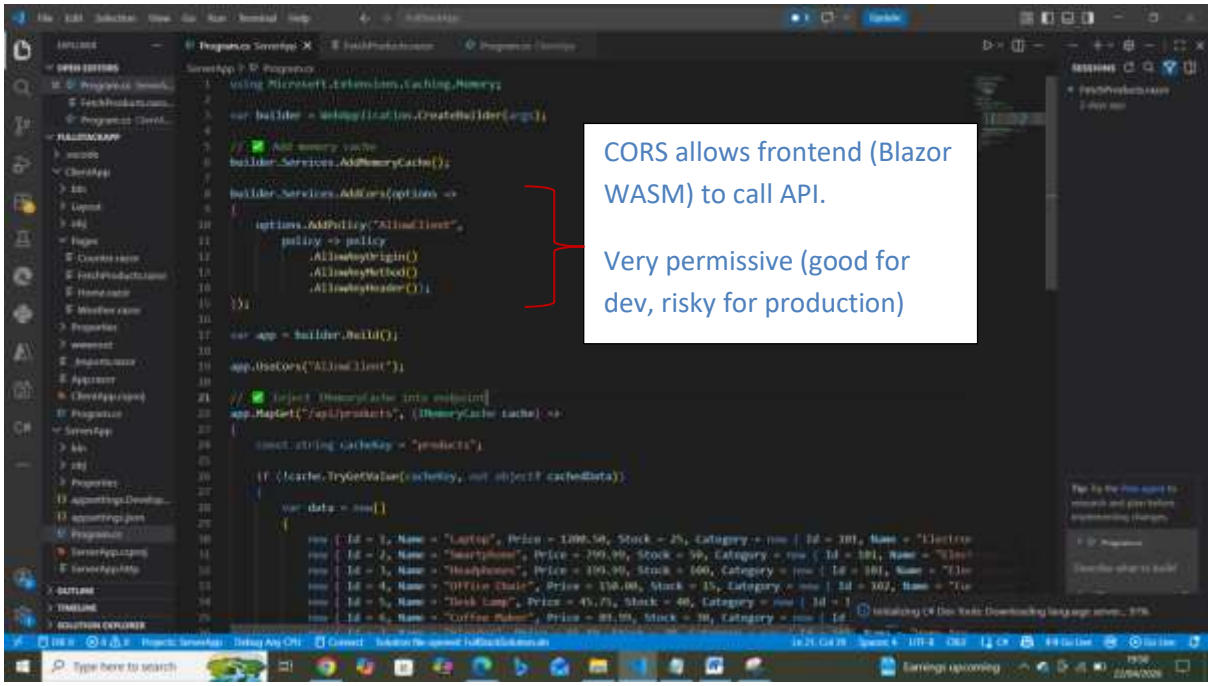
Candidate Name: Kayode Oladipo



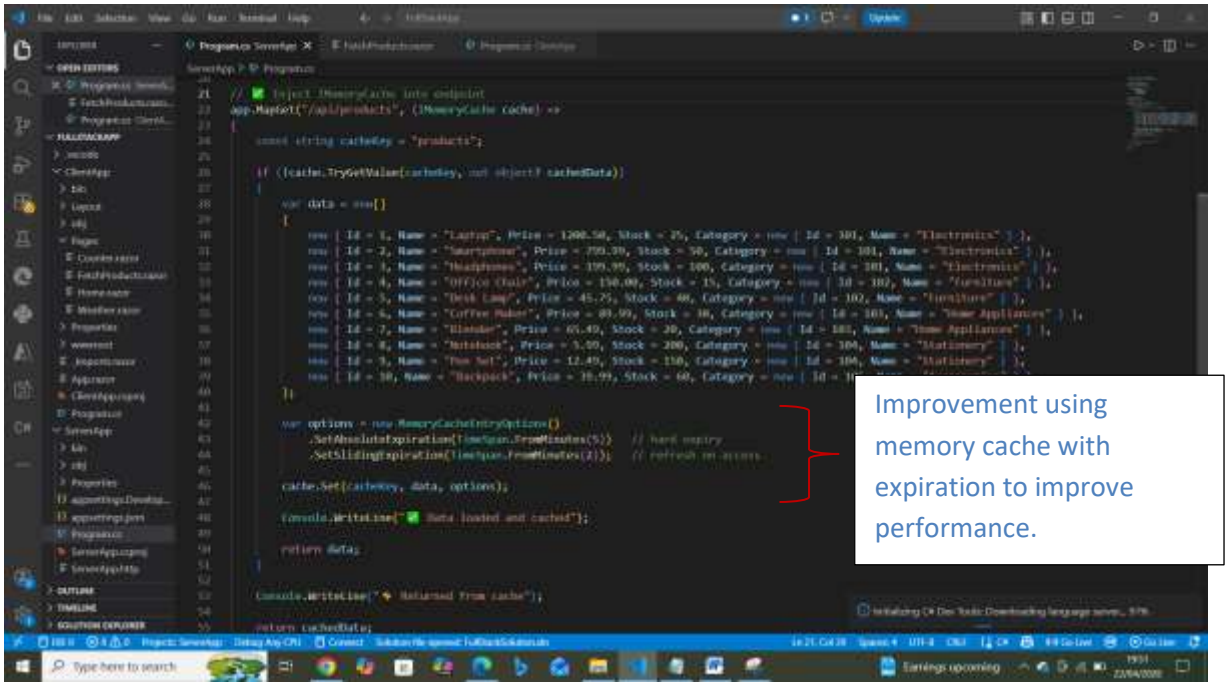


Candidate Name: Kayode Oladipo

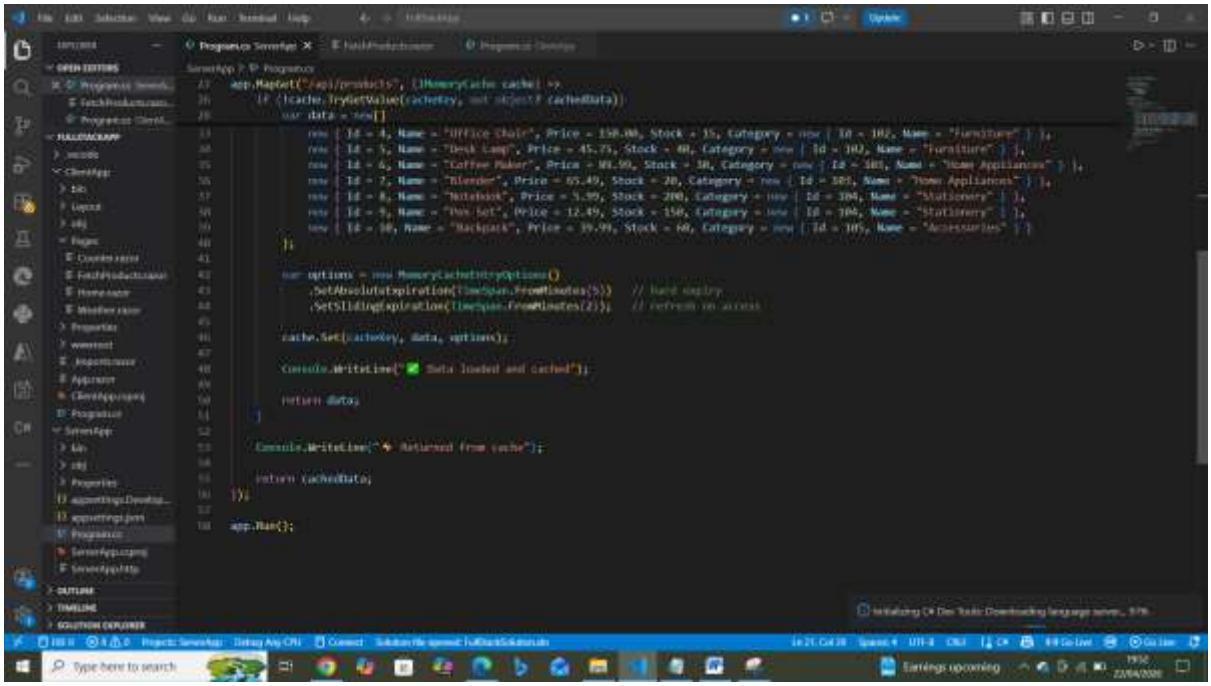
This block of codes shows the creation of the endpoints within Program.cs in the server and using cross resource sharing to allow the client app to use the API from the server. The endpoint is cached for performance.



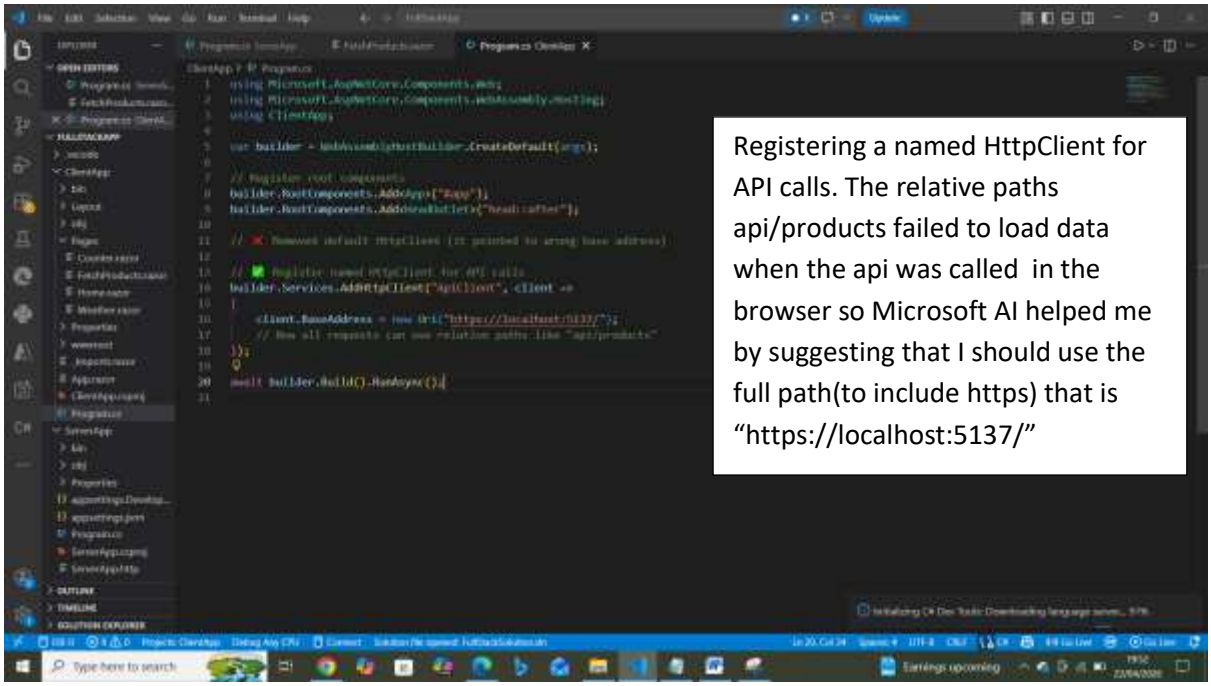
A list is declared here to store the endpoint data



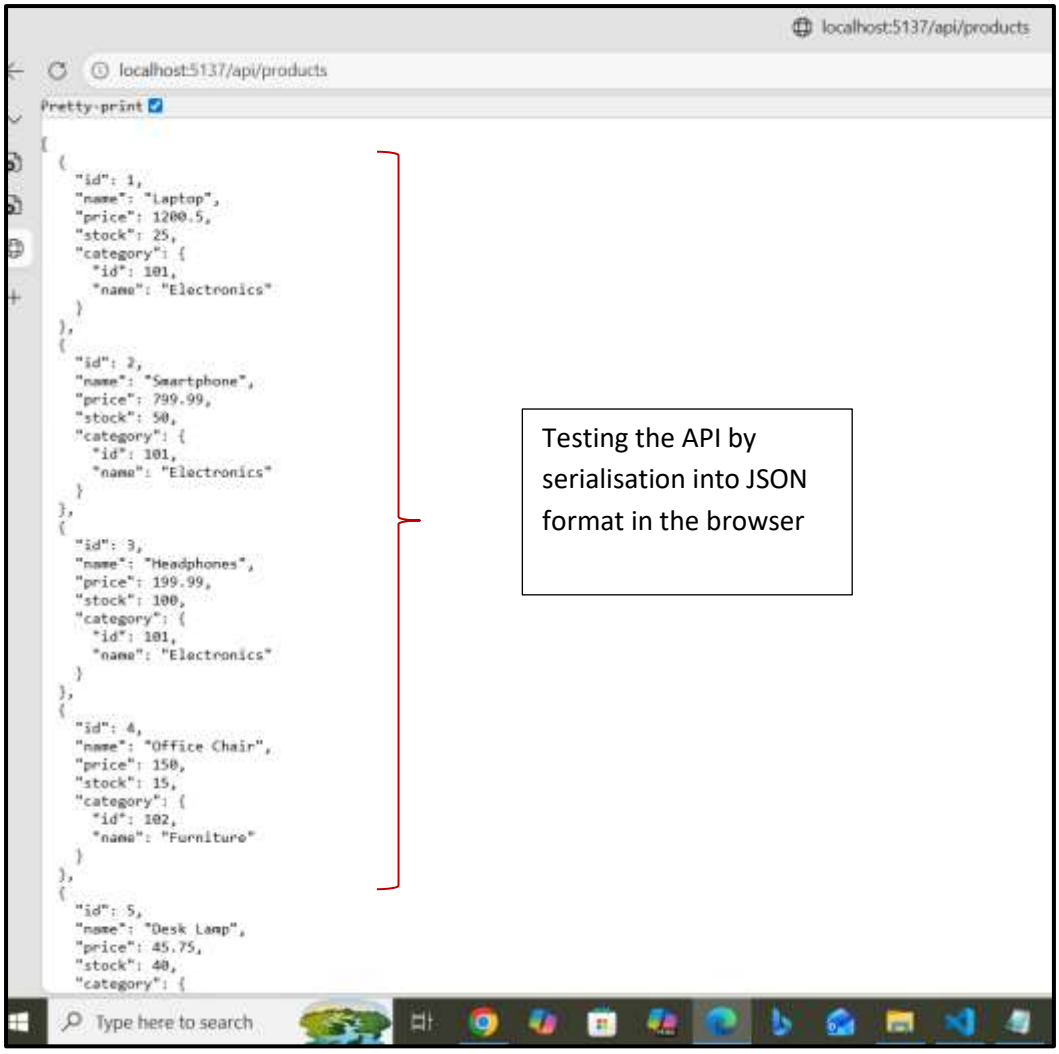
Candidate Name: Kayode Oladipo



Registering the HttpClient for API calls in the Blazor client side within the Program.cs



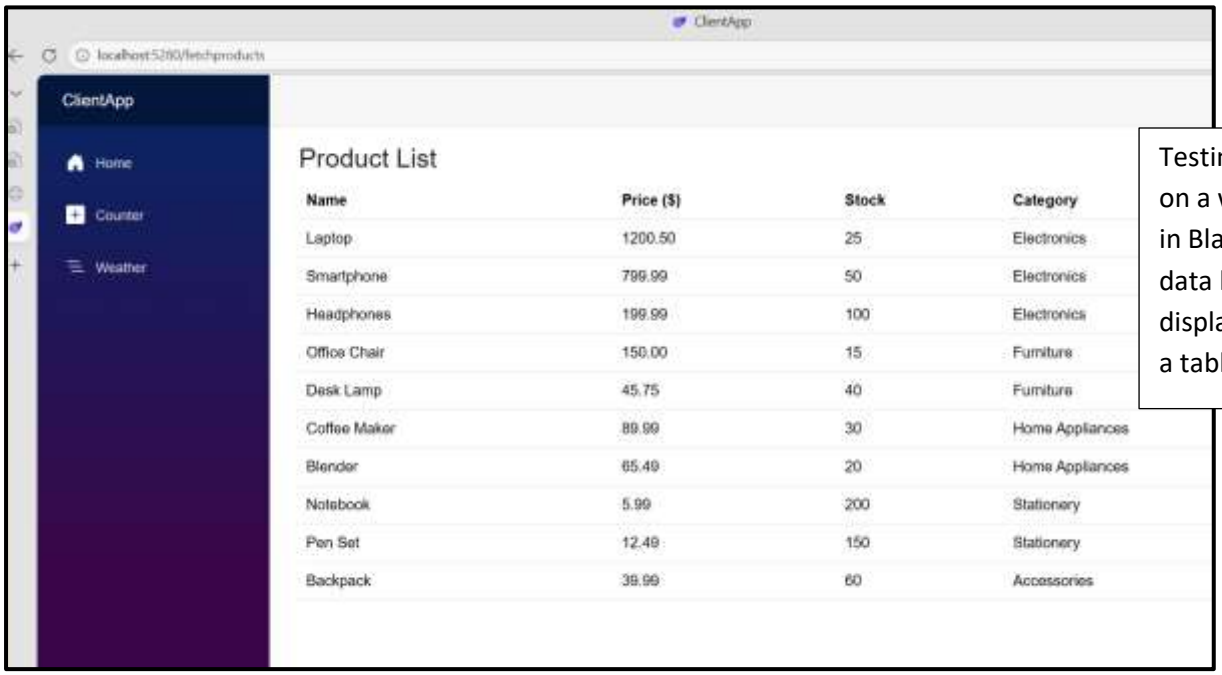
Candidate Name: Kayode Oladipo




```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 1200.5,
    "stock": 25,
    "category": {
      "id": 101,
      "name": "Electronics"
    }
  },
  {
    "id": 2,
    "name": "Smartphone",
    "price": 799.99,
    "stock": 50,
    "category": {
      "id": 101,
      "name": "Electronics"
    }
  },
  {
    "id": 3,
    "name": "Headphones",
    "price": 199.99,
    "stock": 100,
    "category": {
      "id": 101,
      "name": "Electronics"
    }
  },
  {
    "id": 4,
    "name": "Office Chair",
    "price": 150,
    "stock": 15,
    "category": {
      "id": 102,
      "name": "Furniture"
    }
  },
  {
    "id": 5,
    "name": "Desk Lamp",
    "price": 45.75,
    "stock": 40,
    "category": {
```


Testing the API by serialisation into JSON format in the browser

The browser API call in Blazor front end looks like the followings:

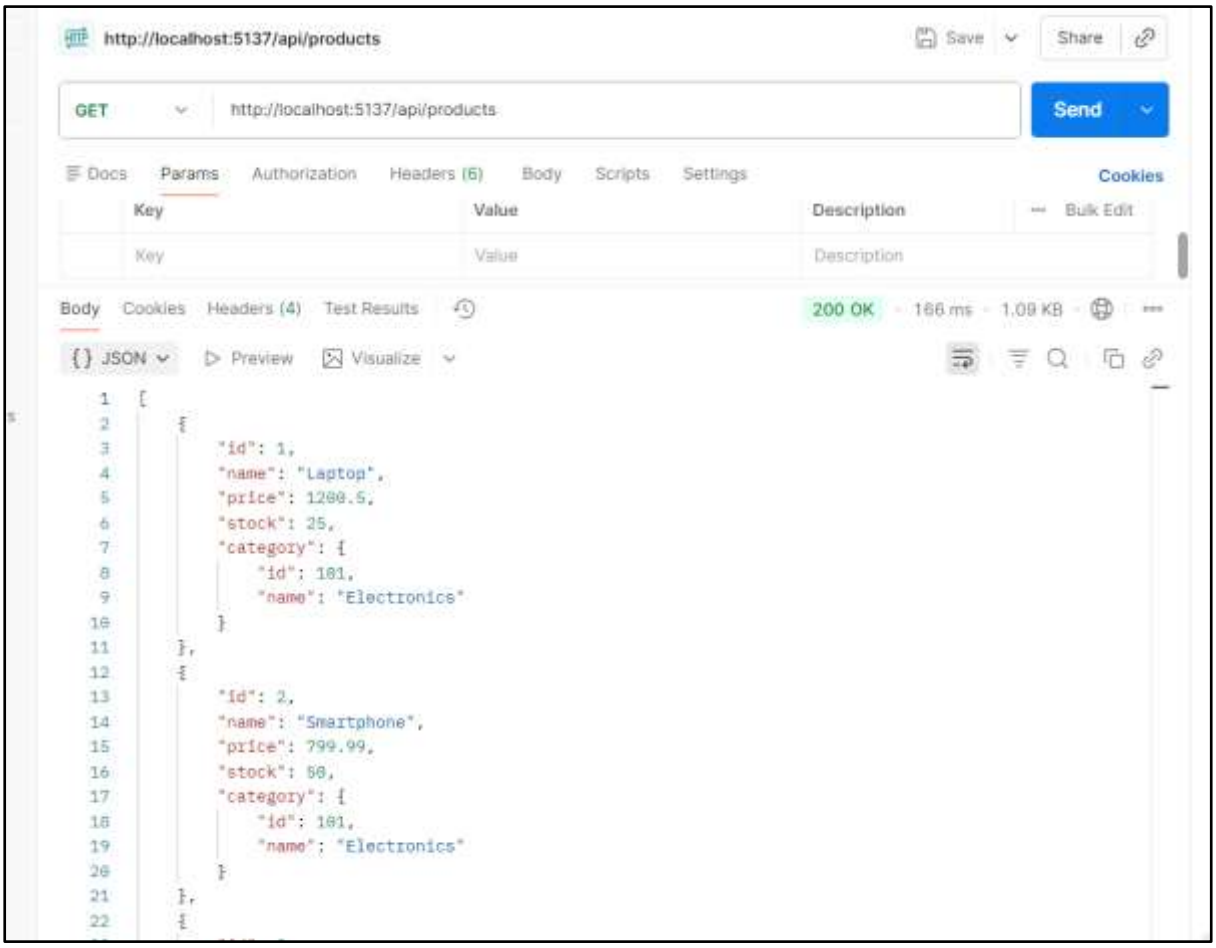


Name	Price ($)	Stock	Category
Laptop	1200.50	25	Electronics
Smartphone	799.99	50	Electronics
Headphones	199.99	100	Electronics
Office Chair	150.00	15	Furniture
Desk Lamp	45.75	40	Furniture
Coffee Maker	89.99	30	Home Appliances
Blender	65.49	20	Home Appliances
Notebook	5.99	200	Stationery
Pen Set	12.49	150	Stationery
Backpack	39.99	60	Accessories

Testing the data on a web page in Blazor. The data has been displayed using a table.

Candidate Name: Kayode Oladipo

Using Postman to test the backend



THE SUMMARY OF THE CODES IS SHOWN BELOW:

```
==============================

CLIENT APP – FetchProducts.razor

==============================

@page "/fetchproducts"
@inject IHttpClientFactory HttpClientFactory

<h3>Product List</h3>

@if (products == null)
{
    <p><em>Loading...</em></p>
}
else if (!products.Any())
{
    <p>No products available.</p>
```

```
        }
        else
        {
            <table class="table">
                <thead>
                    <tr>
                        <th>Name</th>
                        <th>Price ($)</th>
                        <th>Stock</th>
                        <th>Category</th>
                    </tr>
                </thead>
                <tbody>
                    @foreach (var product in products)
                    {
                        <tr>
                            <td>@product.Name</td>
                            <td>@product.Price.ToString("F2")</td> <!-- Format
price -->
                            <td>@product.Stock</td>
                            <td>@product.Category?.Name</td> <!-- Safe navigation -
->
                        </tr>
                    }
                </tbody>
            </table>
        }

        @code {
            // Holds API data
            private List<Product>? products;

            protected override async Task OnInitializedAsync()
            {
                try
                {
                    // ✅ Use named HttpClient instead of hardcoding URL
                    var client = HttpClientFactory.CreateClient("ApiClient");

                    // Fetch data from API
                    products = await
client.GetFromJsonAsync<List<Product>>("api/products");
                }
                catch (Exception ex)
                {
                    // Basic error logging
                    Console.WriteLine($"Error fetching products: {ex.Message}");
                    products = new List<Product>(); // Prevent null issues
                }
            }
```

```
    // ▯ Strongly typed model (better than anonymous objects)
    public class Product
    {
        public int Id { get; set; }
        public string? Name { get; set; }

        // Use decimal for currency (avoids floating-point errors)
        public decimal Price { get; set; }

        public int Stock { get; set; }
        public Category? Category { get; set; }
    }

    public class Category
    {
        public int Id { get; set; }
        public string? Name { get; set; }
    }
}

==============================================
CLIENT APP – Program.cs (Refactored + Commented)
==============================================

using Microsoft.AspNetCore.Components.Web;
using Microsoft.AspNetCore.Components.WebAssembly.Hosting;
using ClientApp;

var builder = WebAssemblyHostBuilder.CreateDefault(args);

// Register root components
builder.RootComponents.Add<App>("#app");
builder.RootComponents.Add<HeadOutlet>("head::after");

// ▯ Removed default HttpClient (it pointed to wrong base address)

// ▯ Register named HttpClient for API calls
builder.Services.AddHttpClient("ApiClient", client =>
{
    client.BaseAddress = new Uri("https://localhost:5137/");
    // Now all requests can use relative paths like "api/products"
});

await builder.Build().RunAsync();

=====================================================
SERVER APP – Program.cs (Refactored + Commented)
=====================================================

using Microsoft.Extensions.Caching.Memory;
```

```
var builder = WebApplication.CreateBuilder(args);

// 🧠 Add memory cache
builder.Services.AddMemoryCache();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowClient",
        policy => policy
            .AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader());
});

var app = builder.Build();

app.UseCors("AllowClient");

// 🧠 Inject IMemoryCache into endpoint
app.MapGet("/api/products", (IMemoryCache cache) =>
{
    const string cacheKey = "products";

    if (!cache.TryGetValue(cacheKey, out object? cachedData))
    {
        var data = new[]
        {
            new { Id = 1, Name = "Laptop", Price = 1200.50, Stock = 25,
Category = new { Id = 101, Name = "Electronics" } },
            new { Id = 2, Name = "Smartphone", Price = 799.99, Stock = 50,
Category = new { Id = 101, Name = "Electronics" } },
            new { Id = 3, Name = "Headphones", Price = 199.99, Stock = 100,
Category = new { Id = 101, Name = "Electronics" } },
            new { Id = 4, Name = "Office Chair", Price = 150.00, Stock =
15, Category = new { Id = 102, Name = "Furniture" } },
            new { Id = 5, Name = "Desk Lamp", Price = 45.75, Stock = 40,
Category = new { Id = 102, Name = "Furniture" } },
            new { Id = 6, Name = "Coffee Maker", Price = 89.99, Stock = 30,
Category = new { Id = 103, Name = "Home Appliances" } },
            new { Id = 7, Name = "Blender", Price = 65.49, Stock = 20,
Category = new { Id = 103, Name = "Home Appliances" } },
            new { Id = 8, Name = "Notebook", Price = 5.99, Stock = 200,
Category = new { Id = 104, Name = "Stationery" } },
            new { Id = 9, Name = "Pen Set", Price = 12.49, Stock = 150,
Category = new { Id = 104, Name = "Stationery" } },
            new { Id = 10, Name = "Backpack", Price = 39.99, Stock = 60,
Category = new { Id = 105, Name = "Accessories" } }
        };

        var options = new MemoryCacheEntryOptions()
```

```
                    .SetAbsoluteExpiration(TimeSpan.FromMinutes(5))   // hard
expiry
                    .SetSlidingExpiration(TimeSpan.FromMinutes(2));   // refresh on
access

                cache.Set(cacheKey, data, options);

                Console.WriteLine("□ Data loaded and cached");

                return data;
            }

            Console.WriteLine("⚡ Returned from cache");

            return cachedData;
        });

        app.Run();

        ==============================================================
```